Race Conditions & The Mutex Mechanism

מבוא:

תכנות מקבילי דורש זהירות יתרה וסנכרון.
בשל מספר סיבות:
- {הפוטנציאל להתרחשות של} race condition
- {הפוטנציאל להתרחשות של} **deadlock** [**קפאון**]
- {הפוטנציאל להתרחשות של} **starvation** [= **הרעבה**]
- סיבות נוספות

תוכן הפרק:

בפרק הזה נדון בנושא של race condition.
נסביר מה פירוש המושג race condition.
נביא דוגמאות קוד שמסבירות את הנושא.
ובהמשך נתאר מנגנון אחד, מתוך כמה מנגנונים, שנועד לאפשר למתכנתים לפתח תוכניות שבמהלך ריצתן לא ייווצר מצב של race condition.

להלן שתי הדוגמאות, מהפשוטה למורכבת.

© יצחק נודלר

דוגמה 1: המונה המשותף (הדגמה בסיסית)

בדוגמה זו, שני חוטים מנסים לקדם (increment) משתנה גלובלי אחד מיליון פעמים כל אחד.
הסטודנטים יצפו לתוצאה של 2,000,000, אך יקבלו תוצאה נמוכה יותר ושונה בכל הרצה.

```
#include <pthread.h>
#include <stdio.h>

/* Shared global variable */
int counter = 0;

/* Function to increment the counter 1,000,000 times */
void* increment_task(void* arg)
{
    for (int i = 0; i < 1000000; i++) {
        /* The Race Condition happens here:
           This operation is not atomic at the CPU level. */
        counter++;
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    /* Create two threads performing the same task */
    pthread_create(&t1, NULL, increment_task, NULL);
    pthread_create(&t2, NULL, increment_task, NULL);

    /* Wait for both threads to finish execution */
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    /* Print the final result */
    printf("Final Counter Value: %d\n", counter);
    printf("Expected Value:      2000000\n");

    return 0;
}
```


© יצחק נודלר

הסבר לוגי מפורט (דוגמה 1)

הסיבה שהקוד נכשל היא בשל העובדה שפעולת ה- ++ אינה **אטומית** (Atomic).
ברמת הוראות המכונה של המעבד, המשפט בשפת C: `counter++` מתורגם לשלושה שלבים נפרדים [= 3 הוראות מכונה !]:

1. **Read:** קריאת הערך מהזיכרון (RAM) לתוך אוגר (Register) במעבד.
2. **Modify:** הוספת 1 לערך בתוך האוגר.
3. **Write:** כתיבת הערך המעודכן מהאוגר חזרה לזיכרון.

הבעיה: כאשר שני חוטים רצים במקביל, ייתכן מצב של "שזירה" (Interleaving):
חוט א' קורא את הערך 100.
באותו רגע, חוט ב' גם קורא את הערך 100.
שניהם מעלים אותו ל-101 באוגרים הנפרדים שלהם, ושניהם כותבים 101 חזרה לזיכרון.

במקום ששני קידומים יביאו אותנו ל-102, איבדנו קידום אחד בגלל ה-"מרוץ" ביניהם.

דוגמה 2: ניהול חשבון בנק (דוגמה מורכבת ומשמעותית)

כאן נדגים מערכת המבצעת אלפי פעולות של הפקדה (Deposit) ומשיכה (Withdraw) במקביל על ידי חוטים שונים. זו דוגמה "מהחיים האמיתיים" שמראה איך באגים כאלו גורמים לנזק כספי או לוגי.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>


/* Structure representing a bank account */
typedef struct {
    double balance;
} BankAccount;

BankAccount my_account = {1000.0}; // Starting with $1000


/* Function to simulate many small deposits */
void* deposit_service(void* arg)
{
    for (int i = 0; i < 100000; i++) {
        double temp = my_account.balance;
        temp += 10.0; // Deposit $10
        /* Simulating a tiny delay to increase the chance of a
context switch */
        my_account.balance = temp;
    }
    return NULL;
}
```


© יצחק נודלר

```
/* Function to simulate many small withdrawals */
void* withdraw_service(void* arg)
{
    for (int i = 0; i < 100000; i++) {
        double temp = my_account.balance;
        temp -= 10.0; // Withdraw $10
        my_account.balance = temp;
    }
    return NULL;
}



int main()
{
    pthread_t t1, t2, t3, t4;

    printf("Starting Balance: $%.2f\n", my_account.balance);

    /* Creating multiple threads for deposits and withdrawals */
    pthread_create(&t1, NULL, deposit_service, NULL);
    pthread_create(&t2, NULL, withdraw_service, NULL);
    pthread_create(&t3, NULL, deposit_service, NULL);
    pthread_create(&t4, NULL, withdraw_service, NULL);

    /* Wait for all banking transactions to complete */
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    /* Theoretically, balance should still be $1000.00 */
    printf("Final Balance:    $%.2f\n", my_account.balance);

    return 0;
}
```

הסבר לוגי מפורט (דוגמה 2)

דוגמה זו מעלה את רמת המורכבות בכמה אופנים:

1. **ריבוי חוטים:** השתמשנו בארבעה חוטים (שניים מפקידים ושניים מושכים). זה מגדיל משמעותית את כמות ה-"התנגשויות" בזיכרון.

2. **מניפולציה מפורשת:** בניגוד ל-++ הפשוט, כאן הפרדנו את הקריאה מהכתיבה על ידי שימוש במשתנה זמני (temp). זה מדמה קוד לוגי אמיתי שבו מבצעים חישובים לפני עדכון הנתונים.

© יצחק נודלר

3. **חוסר עקביות (Non-determinism):** אם נריץ את התוכנית הזו שלוש פעמים, שלוש תוצאות שונות לחלוטין. פעם אחת היתרה תהיה 850$, פעם אחרת 1120$, ולעיתים רחוקות (במקרה) 1000$.

4. **הסכנה:** הסטודנטים צריכים להבין שה-Kernel יכול להחליט להפסיק חוט אחד (Context Switch) בדיוק בין השורה שבה הוא חישב את ה-`temp` לבין השורה שבה הוא עדכן את ה-`balance`. בזמן שהחוט הראשון "ישן", חוטים אחרים שינו את היתרה בזיכרון. כשהחוט הראשון מתעורר, הוא דורס את השינויים של חבריו עם הערך הישן שהיה לו ב-`temp`.

פתרון:

פרופסור, האם תרצה שאציג כעת את הפתרון לבעיות אלו באמצעות מנגנון ה- Mutex (Mutual Exclusion), כדי להראות לסטודנטים איך "נועלים" את הקטע הקריטי?

שלום פרופסור.
כעת לאחר שהצגנו את בעיית ה- race condition, נעבור להציג את הפתרון הקלאסי מעולם ה- System Programming : מנגנון ה-**Mutex** (קיצור של Mutual Exclusion).

מנגנון זה מבטיח שרק חוט אחד יוכל לגשת לקטע קוד מסוים (הקטע הקריטי) בכל רגע נתון.
נשתמש בדוגמה המשמעותית של חשבון הבנק כדי להראות כיצד "נועלים" את המשאב המשותף.

קטע הקוד עם פתרון Mutex

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

/* Structure representing a bank account with its own lock */
typedef struct {
    double balance;
    pthread_mutex_t lock; // The Mutex: A synchronization object
} BankAccount;
```


© יצחק נודלר

```
/* Initialize the account with $1000 and an initialized mutex */
BankAccount my_account = {
    .balance = 1000.0,
    .lock = PTHREAD_MUTEX_INITIALIZER // Static initialization of
the mutex
};



/* Function to simulate many small deposits - NOW THREAD SAFE */
void* deposit_service(void* arg)
{
    for (int i = 0; i < 100000; i++) {
        // [CRITICAL SECTION START]
        // Request exclusive access. If another thread has the
lock, this thread waits.
        pthread_mutex_lock(&my_account.lock);

        double temp = my_account.balance;
        temp += 10.0;
        my_account.balance = temp;

        // [CRITICAL SECTION END]
        // Release the lock so other threads can proceed.
        pthread_mutex_unlock(&my_account.lock);
    }
    return NULL;
}



/* Function to simulate many small withdrawals - NOW THREAD SAFE
*/
void* withdraw_service(void* arg)
{
    for (int i = 0; i < 100000; i++) {
        // Attempt to lock. Only one thread (deposit or withdraw)
can hold it.
        pthread_mutex_lock(&my_account.lock);

        double temp = my_account.balance;
        temp -= 10.0;
        my_account.balance = temp;

        // Release the lock.
        pthread_mutex_unlock(&my_account.lock);
    }
    return NULL;
}
```


© יצחק נודלר

```c
int main()
{
    pthread_t t1, t2, t3, t4;

    printf("Starting Balance: $%.2f\n", my_account.balance);

    pthread_create(&t1, NULL, deposit_service, NULL);
    pthread_create(&t2, NULL, withdraw_service, NULL);
    pthread_create(&t3, NULL, deposit_service, NULL);
    pthread_create(&t4, NULL, withdraw_service, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    /* With Mutex, the result will ALWAYS be exactly $1000.00 */
    printf("Final Balance:    $%.2f\n", my_account.balance);

    // Cleanup: Destroy the mutex when no longer needed
    pthread_mutex_destroy(&my_account.lock);

    return 0;
}
```

הסבר מפורט על מנגנון ה-Mutex

כדי להבין איך הקוד הזה פותר את הבעיה, עלינו לצלול לתוך הקשר שבין ה-C Library לבין ה-Kernel.

4א. מהו ה-Mutex?

ה- Mutex (Mutual Exclusion) הוא אובייקט סנכרון שמתנהג כמו "מפתח" יחיד לחדר.

- כאשר חוט רוצה להיכנס ל**קטע הקריטי** (Critical Section) – אותו אזור בקוד שבו ניגשים למשאב משותף כמו ה- `balance` – הוא חייב לקחת את המפתח באמצעות `pthread_mutex_lock`.
- אם המפתח כבר תפוס על ידי חוט אחר, החוט המבקש **נחסם** (Blocked).

הלוגיקה מנקודת המבט של ה-Kernel

זהו החלק המרתק בהקשר של System Programming:

1. **ניסיון הנעילה:** כשהחוט קורא ל-`pthread_mutex_lock`, הספרייה בודקת (לרוב בעזרת הוראת מעבד אטומית כמו `Atomic Compare-and-Swap`) אם המנעול פנוי.

© יצחק נודלר

2. **המתנה (Wait State):** אם המנעול תפוס, החוט לא מבצע "לולאה אינסופית" (זה היה מבזבז זמן CPU).
במקום זאת, מתבצעת קריאת מערכת (כמו `futex` ב-Linux) שמעבירה את החוט למצב **Sleep/Blocked**.
ה-Kernel מוציא את החוט מתור התזמון (Scheduler) ולא נותן לו זמן מעבד עד שהמנעול משתחרר.

3. **השחרור:** כשהחוט המחזיק במנעול קורא ל-`pthread_mutex_unlock`, הוא "מעיר" את אחד החוטים הממתינים בתור (אם יש כאלו). ה-Kernel מחזיר את החוט שהתעורר למצב `Ready`, והוא יוכל להמשיך בביצוע ברגע שהמתזמן יבחר בו.

ארגומנטים ופונקציות מרכזיות

- **`pthread_mutex_t`:** זהו טיפוס הנתונים של המנעול. הוא מכיל מבנה נתונים פנימי שעוקב אחרי המצב (נעול/פתוח) ואחרי רשימת החוטים הממתינים.

- **`PTHREAD_MUTEX_INITIALIZER`:** זוהי מאקרו לאתחול סטטי. היא קובעת את ערכי ברירת המחדל של המנעול מבלי צורך לקרוא לפונקציית אתחול מפורשת.

- **`pthread_mutex_lock(mutex)`:**
  - *ארגומנט:* מצביע ל-mutex.
  - *פעולה:* ניסיון לקבל בעלות על המנעול. פעולה זו היא **חוסמת** (Blocking).

- **`pthread_mutex_unlock(mutex)`:**
  - *ארגומנט:* מצביע ל-mutex.
  - *פעולה:* שחרור המנעול. זוהי פעולה שחייבת להתבצע תמיד על ידי אותו חוט שביצע את הנעילה.

סיכום הלוגיקה של התכנית המשופרת

בזכות ה-Mutex, המצב שבו שני חוטים מעדכנים את ה-`balance` בו-זמנית הפך לבלתי אפשרי. גם אם ה-Kernel מבצע Context Switch באמצע הפעולה של חוט א', חוט ב' שינסה לרוץ ייעצר מיד בשורת ה-`lock` ולא יוכל לגעת ב-`balance` עד שחוט א' יסיים את עבודתו וישחרר את הנעילה. התוצאה הסופית תהיה תמיד עקבית ומדויקת ($1000.00).

© יצחק נודלר

PTHREAD_MUTEX_INITIALIZER

מהו האובייקט הזה?

`PTHREAD_MUTEX_INITIALIZER` הוא **Macro** (מאקרו) המוגדר בשפת C. הוא משמש כערך קבוע (Constant) המכיל רשימת ערכים ראשוניים עבור המבנה הפנימי של ה-Mutex.

היכן הוא מוגדר?

המאקרו מוגדר בקובץ הכותרת `<pthread.h>`. במערכות מבוססות Linux, קובץ זה הוא חלק מחבילת ה-**Glibc** (GNU C Library).

מהי הגדרתו המדויקת?

ההגדרה משתנה בין ארכיטקטורות (כמו x86_64 לעומת ARM), אך בבסיסה היא נראית כך ב-Glibc:

```
#define PTHREAD_MUTEX_INITIALIZER \
  { { 0, 0, 0, 0, 0, __PTHREAD_SPINS, { 0, 0 } } }
```

הערכים הללו מאתחלים את השדות הפנימיים של ה-Mutex (כמו ה-lock state, ה-owner thread, ותור הממתינים) למצב של "פנוי" (Unlocked) עם הגדרות ברירת מחדל.

האם המימוש הוא ב-Glibc או ב-Kernel?

זוהי נקודה קריטית להבנה:

- **ניהול ה-Mutex:** המנגנון הוא שילוב. ה-**Glibc** (ספריית ה-User-space) מנהלת את רוב הלוגיקה כדי למנוע מעבר מיותר למצב ליבה (Context Switch), מה שחוסך זמן יקר.
- **ה-Kernel:** נכנס לתמונה רק כאשר יש "תחרות" (Contention). אם ה-Mutex תפוס, ה-Glibc מבצעת קריאת מערכת (System Call) מסוג **futex** (Fast Userspace Mutex) כדי לבקש מהליבה להרדים את התהליכון (Thread) עד שה-lock יתפנה.

© יצחק נודלר